■ Custom WordPress Theme Development — Complete Playbook

Converting Next.js + Tailwind + Node.js into Standalone WordPress Themes

Version: 1.0 · July 17, 2026 **Companion to:** WP-Divi5-Dev-PRD-MASTER-COMPLETE.md

> **What This Is:** The complete blueprint for converting any Next.js/Tailwind/Node.js project into a fully functional, standalone WordPress theme you can install on any host. This is the **code-porting path** — distinct from the Divi 5 visual builder path.

■ Table of Contents

---

1. Two WordPress Theme Paradigms

WordPress 2026 supports two theme architectures:

Block Themes (FSE)

- `theme.json` for global styles
- Templates are `.html` files with block markup
- ■ NOT the target for Next.js conversion — too restrictive

Classic Themes (PHP Template Hierarchy) ■

- PHP template files (`index.php`, `header.php`, `single.php`, etc.)
- Full control via `functions.php`
- Supports `wp_enqueue_style/script` for Tailwind
- **This is what you target for code-porting**

---

2. Quick-Start: Pick Your Starter

```
# OPTION A: Roots/Sage (RECOMMENDED — Blade + Tailwind + Vite)
composer create-project roots/sage my-theme
cd my-theme && npm install && npm run build

# OPTION B: _tw (Tailwind without Blade, simpler)
npx @gregsullivan/_tw my-theme

# OPTION C: Bare Underscores (PHP only, no build pipeline)
wp scaffold _s my-theme --theme_name="My Theme" --author="Your Name"
```

Recommendation: Use **Sage** for any production conversion. Blade components are the closest analog to React components. Vite HMR matches Next.js dev experience. Tailwind works identically.

---

3. Starter Themes & Generators Compared

Repository	Stars	Key Features
roots/sage	■13,240	Blade + Tailwind V4 + Vite + theme.json auto-gen
_tw (gregsullivan)	■686	Tailwind + WP-CLI generator + editor integration
wp-tailwind (cjkoepke)	■195	Tailwind + PurgeCSS, classic PHP approach
MountainBreeze	■166	Tailwind + Alpine.js + Vite
Press Wind	■158	Tailwind + Vite, performance-focused
Timberland	■113	Timber (Twig) + ACF + Vite + Alpine.js
Air	■54	Hyper-minimal, Tailwind-only

Underscores (_s) Status in 2026

- Still at `underscores.me` but **stale** (last major update ~2 years ago)
- No Tailwind, no build pipeline
- **Use `_tw` or Sage instead.** Only use _s for absolute minimal PHP baseline.

---

4. Roots/Sage — Architecture Deep Dive

```
my-theme/
■■■ app/                      # PHP: controllers, setup, filters
■   ■■■ Providers/             # Service providers
■   ■■■ View/Composers/        # Data for views (≈ getStaticProps)
■   ■■■ setup.php
■■■ resources/
■   ■■■ views/                 # Blade templates
■   ■   ■■■ layouts/           # Base layouts
■   ■   ■■■ partials/          # Header, footer
■   ■   ■■■ components/        # Reusable Blade components
■   ■■■ css/                   # Tailwind CSS entry
■   ■■■ js/                    # JavaScript
■■■ public/                    # Built assets
■■■ vite.config.js             # Vite + theme.json plugin
■■■ theme.json                 # Auto-generated from Tailwind config
■■■ style.css                  # Theme header
```

Why Sage for Next.js Conversion

1. **Blade ≈ React** — `@props`, `@include`, `@yield`, `@foreach`, `@if`
2. **Tailwind CSS v4** + Vite HMR — identical DX to Next.js
3. **theme.json auto-generates** from Tailwind config
4. **Composers** — data layer mirroring `getStaticProps`/`getServerSideProps`
5. **Service Providers** — dependency injection like Next.js middleware

---

5. Tailwind CSS in WordPress (3 Methods)

Method 1: Vite + Enqueuing (Sage — Recommended)

Sage handles everything via `@roots/vite-plugin`. HMR, Tailwind processing, theme.json — all automatic.

Method 2: Manual Enqueuing

```
// functions.php
function mytheme_enqueue_assets() {
   if (defined('WP_DEBUG') && WP_DEBUG) {
       // Dev: use Vite dev server
       wp_enqueue_script('mytheme-vite', 'http://localhost:5173/@vite/client', [], null);
       wp_enqueue_script('mytheme-app', 'http://localhost:5173/resources/js/app.js', [], null);
   } else {
       // Production: use built manifest
       $manifest = json_decode(file_get_contents(get_template_directory() . '/dist/.vite/manifest.json'), true);
       wp_enqueue_style('mytheme-style', get_template_directory_uri() . '/dist/' . $manifest['resources/css/app.css']['file']);
       wp_enqueue_script('mytheme-app', get_template_directory_uri() . '/dist/' . $manifest['resources/js/app.js']['file']);
   }
}
add_action('wp_enqueue_scripts', 'mytheme_enqueue_assets');
```

Method 3: Tailwind Standalone CLI (No Node.js)

```
curl -sLO https://github.com/tailwindlabs/tailwindcss/releases/latest/download/tailwindcss-windows-x64.exe
./tailwindcss-windows-x64.exe -i ./src/input.css -o ./dist/output.css --watch
```

■■ Critical: Avoid wp-admin Conflicts

```
// tailwind.config.js
module.exports = {
 important: '#app',          // Scope all Tailwind to your theme wrapper
 corePlugins: {
   preflight: false,          // DISABLE CSS reset (conflicts with wp-admin)
 }
}
```

---

6. React/JSX → PHP/Blade Conversion Patterns

Component Mapping

Next.js	WordPress	Notes
`pages/index.tsx`	`front-page.php`	Homepage
`pages/about.tsx`	`page-about.php`	Named page
`pages/blog/[slug].tsx`	`single.php`	Single post
`pages/blog/index.tsx`	`home.php`	Blog listing
`components/Layout.tsx`	`header.php` + `footer.php`	Site chrome
`components/Hero.tsx`	`template-parts/hero.php`	Reusable sections
`components/Card.tsx`	`template-parts/card.php`	UI components
`getStaticProps()`	`WP_Query` or Sage Composer	Data fetching
`next/image`	`wp_get_attachment_image()`	Images
`next/link`	`<a href>` + `get_permalink()`	Links
`{array.map()}`	`@foreach($array as $item)`	Loops
`{condition && <X/>}`	`@if(condition)`	Conditionals
React state (`useState`)	Alpine.js (`x-data`)	Interactivity

JSX → Blade Example

React (Next.js):

```
export function FeatureGrid({ features }) {
 return (
   <section className="py-16 bg-gray-50">
     <div className="container mx-auto px-4">
       <div className="grid md:grid-cols-3 gap-8">
         {features.map((f) => (
           <div key={f.title} className="bg-white p-6 rounded-xl shadow-sm">
             <h3 className="text-xl font-semibold">{f.title}</h3>
             <p className="mt-2 text-gray-600">{f.description}</p>
           </div>
         ))}
       </div>
     </div>
   </section>
 );
}
```

Sage Blade (WordPress):

```
@props(['features' => []])

<section class="py-16 bg-gray-50">
 <div class="container mx-auto px-4">
   <div class="grid md:grid-cols-3 gap-8">
     @foreach($features as $feature)
       <div class="bg-white p-6 rounded-xl shadow-sm">
         <h3 class="text-xl font-semibold">{{ $feature['title'] }}</h3>
         <p class="mt-2 text-gray-600">{{ $feature['description'] }}</p>
       </div>
     @endforeach
   </div>
 </div>
</section>
```

Key: Tailwind classes stay IDENTICAL. Only the logic syntax changes (`map→@foreach`, `condition&&→@if`).

---

7. Next.js Routing → WordPress Template Hierarchy

WordPress resolves URLs by searching for templates in priority order:

Next.js Route	WordPress Template	Notes
`/` → `pages/index.tsx`	`front-page.php`	Falls back to `page.php`
`/blog/` → `pages/blog/index.tsx`	`home.php`	Blog posts listing
`/blog/slug` → `[slug].tsx`	`single.php`	Individual post
`/about` → `pages/about.tsx`	`page-about.php`	Named page template
`/work/`	`archive-work.php`	CPT archive
`/work/slug`	`single-work.php`	CPT single
`/category/slug`	`category.php`	Category archive
`404.tsx`	`404.php`	Not found
Dynamic routes `[param]`	Custom `add_rewrite_rule()`	See below

Custom Rewrite Rules (for dynamic routes)

```
// functions.php — e.g., /work/{slug}
function mytheme_custom_rewrites() {
   add_rewrite_rule(
       '^work/([^/]+)/?$',
       'index.php?post_type=work&name=$matches[1]',
       'top'
   );
}
add_action('init', 'mytheme_custom_rewrites');
```

---

8. WordPress Theme MCP Servers

MCP Server	Tools	Best For
WordPress MCP Adapter (■1,440)	Official Automattic bridge. REST + WP-CLI.	Foundation layer. GitHub
WPVibe MCP (■11)	`create_classic_theme`, file editing, 34 WP-CLI commands.	Theme creation + file management. GitHub
WP-CLI MCP (■3)	45+ tools including theme scaffold, list, install, activate, delete.	Command-line theme control. GitHub
SproutOS (■5)	Full WP control — themes, files, code, DB.	All-in-one. GitHub
WP Clone → Block Theme (■6)	Claude Code skill: clone any website into a block theme.	Quick prototypes. GitHub

---

9. Free ACF in Custom Themes

What Free ACF Gives You

Text, Text Area, Number, Email, URL, WYSIWYG Editor, Image, File, Gallery, Select, Checkbox, Radio, True/False, Date/Time Pickers, Color Picker, Google Map, oEmbed, Relationship, Post Object, Page Link, Taxonomy, User.

Covers ~90% of custom field needs.

What Requires ACF PRO

Repeater, Flexible Content, Options Pages, ACF Blocks, Clone Field.

Bundle ACF as Theme Requirement

```
// functions.php
add_action('admin_notices', function() {
   if (!class_exists('ACF')) {
       echo '<div class="notice notice-warning"><p>';
```

```
        echo 'This theme requires Advanced Custom Fields. ';
        echo '<a href="' . admin_url('plugin-install.php?s=advanced-custom-fields&tab=search') . '">Install ACF</a>';
        echo '</p></div>';
    }
});
```

ACF JSON for Version Control

```
// Save field groups to theme folder
add_filter('acf/settings/save_json', function($path) {
    return get_template_directory() . '/acf-json';
});
add_filter('acf/settings/load_json', function($paths) {
    $paths[] = get_template_directory() . '/acf-json';
    return $paths;
});
```

---

10. Cursor Workflow for Theme Development

Setup Checklist

- [ ] Install WPVibe MCP plugin on WordPress site
- [ ] Configure `.cursor/mcp.json` with WPVibe endpoint
- [ ] Scaffold theme using Sage or `_tw`
- [ ] Run `npm install && npm run dev`
- [ ] Point Cursor at theme folder

Cursor Commands

```
"Create a new classic theme called my-theme with Tailwind support"
"Read the functions.php file and add a custom post type"
"Port this React component to a Sage Blade template"
"Add Alpine.js toggle to this navigation menu"
"Enqueue a custom script in functions.php"
"Run Theme Check on this theme and fix any issues"
```

---

11. Theme Testing & Validation

Theme Check Plugin

WordPress.org — Theme Check — Official automated testing. Same tests used for theme directory submissions.

Theme Unit Test Data

Download from WordPress.org — exhaustive edge-case content (long titles, missing images, floated images, pagination, nested comments, all HTML elements).

WP_DEBUG

```
define('WP_DEBUG', true);
define('WP_DEBUG_LOG', true);    // Logs to wp-content/debug.log
define('WP_DEBUG_DISPLAY', false);
define('SCRIPT_DEBUG', true);     // Unminified CSS/JS
```

PHPCS for WordPress

```
composer require --dev wp-coding-standards/wpcs
./vendor/bin/phpcs --standard=WordPress --extensions=php wp-content/themes/my-theme/
```

---

12. WordPress 7.0 "Armstrong" — Game-Changers for Theme Conversion (July 2026)

> **WP 7.0.2 is current** (released July 17, 2026 — TODAY). Three features directly accelerate the Next.js → WordPress conversion workflow.

PHP-Only Block Registration ■ — THE BIG ONE

Create custom Gutenberg blocks with **zero JavaScript**. This removes the biggest barrier to AI-assisted theme development.

```
// Register a custom block as pure PHP — no JS build step needed
register_block_type('my-theme/project-card', [
   'title'      => 'Project Card',
   'attributes' => [
       'title'   => ['type' => 'string', 'default' => 'Project'],
       'image'   => ['type' => 'integer', 'default' => 0],  // Attachment ID
       'url'     => ['type' => 'string', 'default' => '#'],
   ],
   'render_callback' => function($attributes) {
       $img = wp_get_attachment_image($attributes['image'], 'medium');
       return "<div class='project-card'><a href='{$attributes['url']}'>{$img}<h3>{$attributes['title']}</h3></a></div>";
   },
   'supports' => ['autoRegister' => true], // Auto-generate inspector controls from attributes
]);
```

Why this matters for conversion: When porting React components, you can now create equivalent Gutenberg blocks as PHP without touching Webpack. The `autoRegister` flag auto-generates the sidebar controls. AI can generate these PHP blocks directly.

WP AI Client — Content Migration Assistant

```
// Auto-generate alt text during image migration
$alt_text = wp_ai_client_prompt('Describe this image for a web portfolio: ' . $image_description)
   ->using_model_preference('claude-sonnet-4-6', 'gemini-3.1-pro-preview')
   ->generate_text();

// Generate excerpts from body content
$excerpt = wp_ai_client_prompt('Write a 2-sentence excerpt: ' . wp_strip_all_tags($content))
   ->using_temperature(0.5)
   ->generate_text();
```

Client-Side Abilities API — Hermes Browser Integration

Two packages bridge browser agents to WordPress: - `@wordpress/abilities` — pure state management (works without WordPress) - `@wordpress/core-abilities` — auto-fetches server abilities via REST (`/wp-abilities/v1/`)

Hermes Browser Extension can discover and execute WordPress abilities natively.

Other WP 7.0 Benefits for Conversion

Feature	Benefit for Theme Dev
Block-level custom CSS	Per-block styling without touching theme CSS
Font Library	Central font management — no more `@font-face` hacks
Visual Revisions	Compare before/after conversion states
Responsive block visibility	Show/hide blocks per device
Command Palette (Ctrl+K)	Faster admin navigation

Updated Conversion Workflow With WP 7.0

```
Phase 1: Map Next.js components → PHP-only blocks (no JS needed)
Phase 2: Use WP AI Client to generate alt text, excerpts, meta during content migration
Phase 3: Port React state → Interactivity API (wp_interactivity_state)
Phase 4: Use Font Library for centralized typography
Phase 5: Block-level CSS for per-component styling
Phase 6: Visual Revisions to compare before/after
```

---

13. Deployment — Packaging as .zip

```
cd wp-content/themes/my-theme/
zip -r ../../my-theme.zip . \
 -x "node_modules/*" \
 -x ".git/*" \
 -x "package.json" "package-lock.json" \
 -x "composer.json" "composer.lock" \
 -x "vite.config.js" "postcss.config.js" "tailwind.config.js" \
 -x ".env*" "*.md"
```

Required style.css Header

```
/*
Theme Name: My Theme
```

```
Theme URI: https://example.com
Author: Your Name
Description: Custom theme
Version: 1.0.0
Requires at least: 6.5
Tested up to: 6.9
Requires PHP: 8.0
License: GPL-2.0+
Text Domain: my-theme
*/
```

Upload via wp-admin → Appearance → Themes → Add New → Upload, or:

```
wp theme install my-theme.zip --activate
```

---

13. 8-Phase Conversion Workflow

Phase 1: Analysis (1-2 hrs)

- Inventory all Next.js pages/routes → map to WordPress templates
- Catalog React components → determine Blade/part equivalents
- Identify data sources → plan WordPress data model
- List Tailwind customizations → carry over config

Phase 2: Foundation (2-4 hrs)

- Set up WordPress locally via WP-CLI
- Scaffold with Sage (`composer create-project roots/sage`)
- Install npm dependencies + start Vite watcher
- Copy Tailwind config from Next.js project

Phase 3: Theme Shell (3-6 hrs)

- Port `Layout.tsx` → `header.php` + `footer.php`
- Port navigation → `wp_nav_menu()` with Tailwind walker
- Set up `functions.php` with theme supports
- Copy `globals.css` → theme CSS entry

Phase 4: Page-by-Page (8-20 hrs)

- Convert each page: copy HTML + Tailwind, replace JSX logic with Blade/PHP
- Replace data fetching with `WP_Query` or ACF `get_field()`
- Replace `next/image` → `wp_get_attachment_image()`

Phase 5: Components (4-8 hrs)

- Port React components to Blade components or PHP template parts
- Add Alpine.js for interactivity (toggles, modals, forms)

Phase 6: Data Model (4-8 hrs)

- Set up ACF field groups → export to `acf-json/`
- Register custom post types
- Migrate content from Next.js data source

Phase 7: Testing (4-6 hrs)

- Run Theme Check plugin
- Test with Theme Unit Test data
- Cross-browser + mobile responsive check
- Performance audit (Tailwind purging, image optimization)

Phase 8: Deploy (1-2 hrs)

- `npm run build` for production assets
- Package as .zip

- Upload to WordPress host

Total estimate: 27-54 hours for a full site conversion

---

14. Key GitHub Repositories

Starter Themes

Repository	Stars	Best For
`roots/sage`	13,240	Production Blade/Tailwind/Vite
`gregsullivan/_tw`	686	Tailwind-only, WP-CLI generator
`wpperformance/press-wind`	158	Tailwind + Vite
`cearls/timberland`	113	Twig + ACF + Alpine.js
`joshuaiz/air`	54	Minimal Tailwind

MCP Servers

Repository	Stars	Purpose
`WordPress/mcp-adapter`	1,440	Official MCP bridge
`awesomemotive/wpvibe-ai-mcp`	11	Theme editing via MCP
`mvtandas/wp-cli-mcp`	3	45+ WP-CLI tools
`posimyth/sproutos`	5	Full WordPress MCP

---

15. AI Prompt Templates for Theme Conversion

Initialize Theme

```
Create a new WordPress classic theme called [NAME] using Sage 11.
Set up Tailwind CSS with Vite. The theme should support:
- Custom navigation menus
- Post thumbnails
- HTML5 markup
- ACF integration
- Alpine.js for interactivity
```

Convert a Page

```
Convert this Next.js page to a Sage Blade template:
[PASTE NEXT.JS PAGE CODE]

Map:
- pages/[name].tsx → page-[name].php
- Replace next/image with wp_get_attachment_image()
- Replace next/link with get_permalink()
- Replace getStaticProps with WP_Query
- Keep all Tailwind classes identical
```

Convert a Component

```
Port this React component to a Sage Blade component:
[PASTE REACT COMPONENT]

- {array.map()} → @foreach
- {condition && <X/>} → @if(condition)
- React props → @props
- useState → Alpine.js x-data
- Keep all Tailwind classes identical
```

Playbook v1.0 · July 17, 2026 Research sources: 4 GitHub API searches, wordpress.org plugin repository, roots.io documentation, ACF documentation, Sage v11 docs, MCP server analysis